

目次

研究紹介：グラフニューラルネットワークを用いた物理モデルの自動構築	1
0. 話の流れ	1
1. 目的	2
2. 関連研究	4
3. 手法	7
4. 結果と考察	15
5. まとめ	23

研究紹介：グラフニューラルネットワークを用いた物理モデルの自動構築

発表者：かずひろ

0. 話の流れ

本発表は、研究発表の標準構成に沿って進めます。

1. 目的 — 本研究で何を達成したいか
 2. 関連研究 — この問題は世界でどのように解かれているか
 3. 手法 — どのような方針で解くか（データ・モデル・特徴量）
 4. 結果と考察 — 現時点で何がわかっているか
 5. まとめ — 全体像と今後の展望
-

1. 目的

1.1 物理モデルとは何か

物理モデル：**科学原理**（保存則・反応速度論・輸送現象論など）に基づく数式の組合せで、対象プロセスに関わる物理量の関係を表したもの。

例：連続槽型反応器（Continuous Stirred-Tank Reactor, CSTR）の動特性モデル

$$\frac{dC_A}{dt} = \frac{F}{V}(C_{A,\text{in}} - C_A) - k(T) C_A$$

$$\frac{dT}{dt} = \frac{F}{V}(T_{\text{in}} - T) + \frac{(-\Delta H)}{\rho c_p} k(T) C_A - \frac{UA}{V\rho c_p}(T - T_c)$$

ここで C_A は反応成分 A の濃度、 T は反応器内温度、 F は流量、 V は反応器体積、 $k(T)$ は温度依存の反応速度定数、 ΔH は反応熱、 ρc_p は流体の体積熱容量、 UA は熱交換係数、 T_c は冷媒温度。

これらの数式単体は **変数間の関係式** にすぎず、どの変数を「入力」としてどれを「出力（未知数）」とするかは使用文脈によって決まる。たとえば上記のモデルで運転条件 F, T_{in}, T_c を与えて $C_A(t), T(t)$ の時間発展を解くこともできるし、所望の C_A を得るための F を逆算することもできる。

1.2 誰が物理モデルを必要とするか

定性的な説明（「温度を高めにして十分に反応させる」）では不十分で、**定量的な予測・判断が必要となる場面**がある。代表例：

- **プロセス設計**：パイロット装置から実機へのスケールアップで、反応器体積・熱交換器面積・流量などの運転条件を決定する
- **制御系設計**：プラントの動特性を把握したうえで、**制御構造**（フィードバック・フィードフォワード等）の選定、制御器の設計と性能評価を行う
- **プロセス最適化**：収率・エネルギー消費・環境負荷などを数値的に表す **目的関数を定義** し、制約条件（安全・品質・機器仕様）のもとで、運転条件や設計変数を最適化する

- ・ **プロセス安全**：異常運転・起動停止時・外乱発生時の挙動を事前にシミュレーションし、安全装置の仕様や運転範囲を決定する（例：反応熱除去が追いつかなくなる「**暴走反応**」の回避）

これらの業務では、**計算可能な形で記述された数式体系**（物理モデル）そのものが不可欠な成果物となる。

1.3 現状の課題

既存プロセス・典型的なユニット操作であれば、**プロセスシミュレータ**（Aspen Plus, gPROMS, MATLAB/Simulink など）に内蔵された標準モデルを選択・結合することでモデル構築できる。しかし、次のような場面では依然として専門家の手作業が必要となる：

- ・ **新規プロセス**：シミュレータの標準ライブラリに該当モデルがない場合
- ・ **複数分野の連成**：反応・伝熱・輸送現象・制御を跨ぐ新しい組合せモデル
- ・ **文献ベースの再現**：特定の論文に記載された式を用いて他者の結果を再現・比較したい場合

これらでは、教科書や論文を調査し、必要な式を選別して連立する作業を人手で行う。典型的な所要時間は、対象の複雑さと熟練度に依存するが、**数日から数週間のオーダー**である。熟練者の経験に依存し、再現性や効率の観点から課題となっている。

1.4 本研究の目的

上記の手作業部分を自動化することが本研究の目的である。具体的には、

プロセスの自然言語記述と求めたい変数の情報から、文献中の実在数式データベースを検索し、連立可能な数式群を自動選択する手法を確立する。

重視する要件は 2 点：

- ・ **ハルシネーションの排除**：出力される数式は全て実在し、出典（文献・教科書のページ）を辿れる
- ・ **自由度的に解ける数式群**：個別の式を寄せ集めるのではなく、選ばれた式群を連立したときに**未知数の数と独立な式の数が一致する（自由度＝ゼロ）**ことを保証する。この条件を満たさない式の集合は、未知数が過剰で解が一意に定まらない、あるいは独立式が過剰で解自体が存在

しない

2. 関連研究

物理モデルの自動構築に関する関連研究は、大きく 3 流派に整理できる。

2.1 流派 A：シンボリック回帰 (Symbolic Regression)

入出力データの数値ペア $\{(x_i, y_i)\}$ (x_i は多変数ベクトル) から、 $y \approx f(x)$ となる数式 f を探索する手法群。遺伝的プログラミング (例: PySR, Eureqa) や、Feynman 方程式の再発見を目指す AI Feynman (Udrescu & Tegmark, 2020) が代表例。

必要な実験データ量の目安: AI Feynman はファインマン物理学教科書の 100 本の公式を探索で再発見したが、その際 1 公式あたり 10^5 点 (10 万組) 程度の入出力ペアを用いている。例えばニュートンの万有引力則 $F = Gm_1m_2/r^2$ を再発見する場合、3 つの入力 (m_1, m_2, r) と出力 F の組を 10 万組生成して探索を実行する。より多変数・より非線形な系では 10^6 点以上が必要となることも珍しくない。このデータ量は実験室で人手により収集するには非現実的であり、大規模シミュレーションや十分な測定データの存在が前提となる。

- **長所:** 実験データから未知の関係式を発見できる可能性がある
- **短所:**
 - 実験データが前提のため、設計段階や新規プロセスでは利用困難
 - 既知の物理法則や文献知識を明示的に取り込みにくい
 - 探索空間が広く、得られた式が工学的に意味のある形 (保存則・無次元化された形式など) に収束するとは限らない (数式検索方式は既存式を選ぶため最初から意味ある形式で得られる、大規模言語モデル方式は特定形式への誘導をプロンプトで指示できる、のに対しシンボリック回帰は数値誤差最小化を目的とするため形式制御が間接的)

2.2 流派 B：大規模言語モデル（Large Language Model, LLM）による直接生成

大規模言語モデル（GPT-4, Claude 等）に自然言語でプロセス記述を与え、数式モデルを直接生成させる手法。近年研究例が増えている：

- Bran ら (2023) の **ChemCrow**：大規模言語モデルに化学反応予測と合成計画を直接行わせる
- Ramos ら (2024)：反応器モデルの常微分方程式（Ordinary Differential Equation, ODE）を大規模言語モデルで直接記述させ、Python シミュレーションコードまで生成
- **長所**：自然言語で指示でき、応答は**数秒～数十秒**のオーダー
- **短所**（比較対象は項目ごとに明記する）：
 - **ハルシネーション（対数式検索方式）**：数式検索方式は文献中の実在式のみを返すため原理的に架空の式は出ない。これに対し大規模言語モデルは存在しない式や定数を捏造した式を出力する可能性があり、**モデル自身はその正誤を判定できない**
 - **根拠追跡の困難（対数式検索方式）**：数式検索方式は選ばれた式が文献中の特定の式と 1 対 1 に対応するため、出典（書名・論文・式番号）を辿れる。大規模言語モデル直接生成では、出力された式が既存式の引用か、組み合わせか、新規構成かを区別できず、出典追跡ができない
 - **組合せ整合性の不安定性（対シンボリック回帰・数式検索方式）**：数式検索方式は正解集合の式を選ぶ問題として自由度条件（§1.4）を明示的に検査できる。シンボリック回帰は数値フィッティングで単一の式を得るため組合せ問題は発生しない。大規模言語モデル直接生成は連立系として扱う明示的な仕組みがなく、未知数の数と独立式の数が合わない系を出力する場合がある

2.3 流派 C：文献からの数式検索（本研究の位置づけ）

着想 — 数式データから SNS へ、SNS からグラフへ：本研究の出発点は、**数式データそのものの見え方**から得た素朴な直感である。数式は一つ一つが独立した個体として存在する一方で、互いに**変数共有・共起・入出力連鎖**（具体的な仮説は §3.3.1 で詳述）といった関係で繋がっている。このような「個体 + 関係」で構成されるデータを眺めたときに最初に想起されたのが、**ソーシャルネット**

ワーク (SNS) である。

SNS	本研究の数式ネットワーク
人物	数式
友人関係	共通変数で結ばれる関係
$A \leftrightarrow B \leftrightarrow C$	式 e_1 - 変数 T - 式 e_2 (変数 T を両方が含む)

A さんと C さんに直接の接点がなくとも共通の友人 B がいれば関係を推測できるように、式 e_1 と e_2 が直接変数を共有していなくても、共通変数ノード（あるいは共通の式）を介した **2 ホップ先の関係**から「同じ問題設定で連立される可能性」を推測できる。ここから推論は自然に繋がる：SNS とは要するに **グラフ**（ノード = 人物、エッジ = 友人関係）であり、グラフ上でノードの特徴量と関係性を同時に扱う代表的な学習手法と言えば、**グラフニューラルネットワーク (Graph Neural Network, GNN)** である。こうして本研究は、流派 C（文献からの数式検索）+ GNN 的アプローチ、という方向へ向かうことになった。

流派 C の定義：教科書・論文から抽出した**実在する数式のデータベース**を構築し、そこから条件に合う数式群を**選び出す**手法。「生成」ではなく「検索と選択」として問題を定式化する。

- **長所**：

- 提示される式が全て実在し、出典を辿れる（ハルシネーションが原理的に起こらない）
- 「正解集合」を明示的に定義できる：出力が**式 ID の集合**なので、正解 ID 集合との積集合・和集合で直接比較できる。一方、シンボリック回帰・大規模言語モデル方式の出力は**数式表現そのもの**であり、同値な数式の書き方が複数あるため（例： $y = 2x$ と $y = x + x$ ）、自動比較には数式の等価性判定が必要となる

- **課題**：

- 選択の範囲はデータベース収録式に限定される（新規の式は発見できない）
- シンボリック回帰との相補関係：既存式から組み立てたい場合 → 数式検索、既存式で説明困難な現象 → シンボリック回帰

本研究は流派 C に位置づけられる。現在、3,244 の実在数式を収録したデータベースを構築している（詳細は §4.1）。

3. 手法

3.1 問題設定

- 入力：

- プロセスの自然言語記述（例：「連続槽型反応器で $A \rightarrow B$ の反応を行う。反応は発熱で、ジャケットで冷却する」）
- 既知変数の集合 V_{in} （入力として与える物理量の記号、例： $\{F, T_{\text{in}}, C_{A,\text{in}}\}$ ）
- 求めたい変数の集合 V_{out} （出力として解きたい物理量、例： $\{C_A, T\}$ ）

- 出力：

- V_{in} と連立して V_{out} を解けるような、データベース内の数式の集合

3.2 基本戦略：2 段階の数式検索

データベースには数千の式が収録されており、全ての式について詳細な整合性評価を行うのは計算量の観点で非現実的である。そこで、情報検索で標準的な **2 段階アーキテクチャ** を採用する：

段階	処理	役割	計算量
Stage 1	文脈類似度（TF-IDF）＋ 変数一致度（Jaccard）で上位 K 件を選抜	大域的な絞り込み	$O(N)$
Stage 2	Stage 1 の K 件を精密に評価して並べ替え	詳細な選択	$O(K), K \ll N$

本研究では $K = 50$ を採用している。Stage 1 だけでも正解式が上位 50 件に含まれる割合 (**Recall@50**) は 99% 以上であり、Stage 2 の前段として十分機能する。**研究の中心は Stage 2**（候補式群の精密評価）であり、ここにグラフ構造の情報を組み込む。



■3.2.1 Stage 1 スコアの具体的な計算式 Stage 1 のスコアは以下の 2 成分の重み付き和で与える。いずれも古典的な統計手法であり、大規模言語モデルは使用しない（同一入力に対して完全に決定論的）。

(a) TF - IDF による文脈類似度

文書（クエリ q のプロセス記述、または式 e の周辺説明文）を、単語頻度（TF: term frequency）と逆文書頻度（IDF: inverse document frequency）の積で重み付けしたベクトルに変換する：

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)}$$

- $\text{tf}(t, d)$ ：文書 d における用語 t の出現回数
- $\text{df}(t)$ ：用語 t を含む文書数、 N ：全文書数（本研究では $N = 3,244$ ）
- 直感的には「ある文書に何度も現れるが、他の文書にはあまり現れない語」が大きな重みをもつ

文書ベクトル $\mathbf{v}_d \in \mathbb{R}^{|V|}$ （ V は語彙集合）同士のコサイン類似度で文脈の近さを測る：

$$\text{sim}_{\text{text}}(q, e) = \cos(\mathbf{v}_q, \mathbf{v}_e) = \frac{\mathbf{v}_q \cdot \mathbf{v}_e}{\|\mathbf{v}_q\| \|\mathbf{v}_e\|}$$

実装は scikit-learn の `TfidfVectorizer`（unigram + bigram、最大語彙 50,000、smoothed IDF）を用いる。

(b) Jaccard による変数一致度

クエリの入出力変数集合 $V_q = V_{\text{in}} \cup V_{\text{out}}$ と、式 e が含む変数集合 V_e の記号レベルの重なりを Jaccard 係数で測る：

$$\text{sim}_{\text{var}}(q, e) = J(V_q, V_e) = \frac{|V_q \cap V_e|}{|V_q \cup V_e|}$$

例： $V_q = \{F, T_{\text{in}}, C_{A, \text{in}}, C_A, T\}$ 、 $V_e = \{C_A, T, F, V\}$ のとき $|V_q \cap V_e| = 3$ 、 $|V_q \cup V_e| = 6$ なので

$J = 0.5$ 。

(c) 統合スコア

両者の線形結合で Stage 1 スコアを定義する：

$$S_{\text{Stage1}}(q, e) = \alpha \cdot \text{sim}_{\text{text}}(q, e) + (1 - \alpha) \cdot \text{sim}_{\text{var}}(q, e)$$

検証データ上のグリッドサーチで最適となった $\alpha = 0.7$ を採用する。 S_{Stage1} の降順で上位 $K = 50$ 件を Stage 2 に渡す。

注記：大規模言語モデルは**データ構築フェーズ**（式抽出・訓練ケース生成、§4.1 参照）でのみ使用し、検索実行時（Stage 1・Stage 2）には一切関与しない。

3.3 グラフ構造とモデルの入出力

■3.3.1 **なぜグラフ構造を用いるか** 数式データには、各式のテキストを単独で見ているだけでは捉えきれない**式と式の関係性**が豊富に含まれている。本研究では、以下の3つの**作業仮説**に基づき、グラフ構造による表現が有用であると考ええる。

1. **変数共有**：同じ変数を用いる式同士は、同一の問題設定のなかで一緒に連立される可能性が高い。ただし、共有変数の**希少度**によって情報量は大きく異なる点に注意が必要である：

- 温度 T 、体積 V 、圧力 P のような **基礎変数**はほぼ全分野に登場するため、これらを共有するだけでは弱い手がかりにしかない（例：理想気体状態方程式 $PV = nRT$ と Arrhenius 式 $k = A \exp(-E_a/RT)$ は共に T を含むが、独立な現象を記述する）。
- 一方、抗力係数 C_d 、ヌセルト数 Nu 、プラントル数 Pr のような **分野固有の希少変数**を共有する式は、高い確度で同じ分野（流体力学、伝熱工学等）の問題に属する。

したがって本項はより正確には「**共有変数の希少度で重み付けた変数一致度**が、連立される可能性と相関する」という仮説であり、実験で検証すべき対象である。

2. **共起**：同じ文献・同じドメインで頻繁に共起する式の組は、問題設定として一緒に連立される可能性が高い。これは（1）と相関するが、変数の記号表記が文献間で揺れる場合（同じ物理

量が別の記号で書かれる場合)にも独立に機能するため、補完的な観測軸となる。

3. **入出力連鎖**：ある式の出力変数が別の式の入力変数として必要になる場合、両者は連立しなければ閉じた系を構成できない可能性が高い。これは(1)の「変数共有」に方向性を付与した関係であり、自由度解析の基礎そのものになる。

これらの関係はいずれも**ノード(式・変数)**と**エッジ(包含・共起・依存)**で表現でき、単一式のテキスト表現では直接扱えない。したがって、テキスト特徴(TF-IDF, CodeBERT等)にグラフ特徴を補完的に組み合わせる設計が妥当と考える。

ただし上記はあくまで作業仮説であり、グラフ特徴量が実際に retrieval 性能を改善するかは実験で検証する必要がある (§4.2)。実際、初期研究で試した本格的なグラフニューラルネットワーク(グラフ畳み込みネットワーク、グラフ注意ネットワーク)は単体ではベースラインを超えられず、現在は「グラフ特徴量 + 多層パーセプトロン」の形に落ち着いた経緯がある (§3.3.4 詳述)。なお、グラフ化の具体的方法(二部グラフ/ハイパーグラフ等)の選択については §3.3.2 の注記で述べる(SNSを用いた直感的な動機については §2.3 を参照)。

■3.3.2 **グラフの構成** 本研究では、数式間の関係を **二部グラフ** として表現する：

- **ノード**：2 種類
 - 数式ノード： $N_{\text{eq}} = 3,244$ 個
 - 変数ノード： $N_{\text{var}} \approx 3,700$ 個(同一記号は同一ノードとして扱う)
- **エッジ**：式 e が変数 v を含む場合に、 e と v を結ぶ無向エッジを張る
- **ノード特徴ベクトル**：
 - 数式ノード：式のテキスト(LaTeX 文字列)と周辺の説明文を CodeBERT で埋め込み、768 次元ベクトルとする
 - 変数ノード：該当変数を含む全ての式ノード特徴の平均

注記：数式間の関係をグラフで表す方法は一意ではない(例：ハイパーグラフ、知識グラフ等)。本研究では「変数共有」という最も基本的な関係に着目し、上記の二部グラフを採用している。



■3.3.3 Stage 2 リランカーの入出力 Stage 2 では、Stage 1 で選抜された $K = 50$ 件の候補式 1 件ごとに「特徴ベクトル → 多層パーセプトロン → スカラスコア」の順で計算する。

(a) 特徴ベクトル $\phi(q, c)$ の構成 (入力)

候補式 c とクエリ q から 10 次元の特徴ベクトルを作る：

$$\phi(q, c) = [\phi_1, \phi_2, \dots, \phi_{10}]^\top \in \mathbb{R}^{10}$$

各要素 ϕ_i の定義は §3.3.4 の表に従う。代表例のみ再掲：

$$\phi_1 = \text{sim}_{\text{text}}(q, c), \quad \phi_2 = J(V_q, V_c), \quad \phi_3 = \text{sim}_{\text{svd}}(q, c)$$

(sim_{text} と J は §3.2.1 の定義、 sim_{svd} は TF-IDF を 256 次元に圧縮した SVD 空間での余弦類似度)。

(b) 多層パーセプトロン (モデル本体)

隠れ層 2 層・各 64 次元の多層パーセプトロンで、10 次元の特徴を 1 次元のスコアに写像する：

$$\begin{aligned} \mathbf{h}_1 &= \text{ReLU}(W_1 \phi(q, c) + \mathbf{b}_1) \in \mathbb{R}^{64} \\ \mathbf{h}_2 &= \text{ReLU}(W_2 \mathbf{h}_1 + \mathbf{b}_2) \in \mathbb{R}^{64} \\ s(q, c) &= \mathbf{w}_3^\top \mathbf{h}_2 + b_3 \in \mathbb{R} \end{aligned}$$

- 学習パラメータ： $W_1 \in \mathbb{R}^{64 \times 10}$ 、 $W_2 \in \mathbb{R}^{64 \times 64}$ 、 $\mathbf{w}_3 \in \mathbb{R}^{64}$ 、バイアス $\mathbf{b}_1, \mathbf{b}_2, b_3$
- 活性化関数： $\text{ReLU}(x) = \max(0, x)$
- 各 ReLU の後に **ドロップアウト** (確率 $p = 0.1$) を適用。訓練時のみ有効で、評価時には外す。

ドロップアウトについて (補足)：過学習を防ぐための正則化手法。隠れベクトル $\mathbf{h}$ の各要素 h_i を独立に確率 p で 0 に置き換え、残った要素は期待値保存のため $1/(1-p)$ 倍に拡大する：

$$\tilde{h}_i = \frac{m_i}{1-p} \cdot h_i, \quad m_i \sim \text{Bernoulli}(1-p)$$

$p = 0.1$ ならば平均して隠れベクトル 64 次元のうち約 6~7 個の要素が 0 になり、残りが $1/0.9 \approx 1.11$ 倍に拡大される。訓練中は毎回異なる「部分ネットワーク」で予測するため、モデルが特定の隠れユニットに過度に依存できなくなる（実質的なアンサンブル効果）。評価時はマスクを外して全ユニットをそのまま用い、決定論的に推論する。本研究では訓練データが約 1,100 ケースと比較的少ないため、ドロップアウトは過学習抑制に重要な役割を果たす。

(c) 出力スコアとランキング

$s(q, c) \in \mathbb{R}$ は**ランキング用の生スコア**（ロジット）であり、確率値ではない（シグモイドは通さない）。最終的な並び順は候補集合 $\{c_1, \dots, c_{50}\}$ を $s(q, c)$ の降順でソートして決まる：

$$\hat{c}_{(1)}, \hat{c}_{(2)}, \dots, \hat{c}_{(50)} = \underset{c \in \text{Stage 1 上位 50}}{\operatorname{argsort}} (-s(q, c))$$

(d) 訓練の損失関数（参考）

正解候補 c^+ と不正解候補 c^- のペアについて **pairwise margin ranking loss**（ヒンジ損失）を最小化する：

$$\mathcal{L} = \sum_q \sum_{(c^+, c^-)} \max(0, m - s(q, c^+) + s(q, c^-)), \quad m = 0.1$$

これは「正解候補のスコアを、不正解候補のスコアより少なくともマージン m だけ高くする」ことを要求する損失である。二値分類の BCE（sigmoid + cross-entropy）ではなく、順序関係のみを学習するランキング専用の損失を用いるため、出力スコア $s(q, c)$ は確率値として解釈しない。

重要：本研究で用いるモデルは厳密には多層パーセプトロンであり、**メッセージパッシング型**のグラフニューラルネットワーク（Graph Neural Network, GNN）そのものではない。ただし、特徴ベクトル $\phi(q, c)$ の一部（ $\phi_8, \phi_9, \phi_{10}$ ）は**二部グラフ上の近傍情報から計算されるグラフ特徴量**であり、グラフ構造の情報がモデルに入力されている点で、「グラフニューラルネットワーク的アプローチ」と位置づけている。初期研究では本格的なグラフニューラルネットワーク（グラフ畳み込みネットワーク GCN、グラフ注意ネットワーク GAT）も試みたが、ベースラインを超えられなかった経緯があり、現状は上記の設計に収束している。



■3.3.4 特徴ベクトルの内訳 (10 次元)

分類	#	特徴量	意味
基本 (7 次元)	1	TF - IDF 余弦類似度	クエリ文脈と式テキスト のテキスト類似度
	2	変数集合の Jaccard 係数	クエリ I/O 変数と式の 変数の記号一致率
	3	SVD 空間の余弦類似度	TF - IDF を 256 次元 SVD に圧縮した意味類 似度
	4	入力変数カバー率	クエリ入力変数のうち式 に含まれる割合
	5	出力変数カバー率	クエリ出力変数のうち式 に含まれる割合
	6	逆カバー率	式の全変数のうち、クエ リ I/O に含まれる割合
	7	ドメイン一致	式のドメインがクエリ文 脈と一致するか
Set-aware (3 次元)	8	補完性	候補が上位 K' 件ではま だカバーされていないク エリ変数をどれだけ補 うか
	9	一貫性	候補の変数が上位 K' 件 とどれだけ共有されてい るか
	10	ドメイン整合性	候補のドメインが上位 K' 件のドメインと一致 する割合

ここで「上位 K' 件」は Stage 1 で得られたスコア上位 $K' = 5$ 件の候補集合を指す (§3.5 で詳述)。

3.4 単発選択から組合せ選択へ

本研究の正解集合は 1 式とは限らない。訓練ケースの約 44% は正解が 2 式以上である (§4.1)。

単発選択型の特徴量 (候補 1 件ごとに独立にスコアリング) だけでは、組合せとしての整合性は評価されない。具体的には、次のような場面で不足が生じる：

- 候補 A と B が別々に評価されて両方高スコアだが、両者は同じ変数しかカバーしておらず、連立してもクエリの全出力変数が解けない
- 候補 A はドメイン X、候補 B はドメイン Y と評価されたが、両者が組み合わさって使われた先例が文献上ない

このような失敗を避けるため、**候補同士の組合せを評価する特徴量**を導入する必要がある。

3.5 Set-aware 特徴量の定義

候補集合の整合性を評価するため、Stage 1 スコアで上位 $K' = 5$ 件の候補を **参照集合** S_{ref} とし、各候補 c について次の 3 特徴量を計算する：

1. **補完性 (Complementarity)**： 候補 c がクエリの入出力変数 V_{query} のうち、 S_{ref} ではカバーされていない変数をどれだけ補うか。

$$\text{Comp}(c) = \frac{|(V_{\text{query}} \cap V_c) \setminus V_{S_{\text{ref}}}|}{|V_{\text{query}}|}$$

ここで V_c は c の変数集合、 $V_{S_{\text{ref}}}$ は S_{ref} の全変数の和集合。

2. **一貫性 (Coherence)**： c の変数のうち、 S_{ref} と共有する割合。

$$\text{Coh}(c) = \frac{|V_c \cap V_{S_{\text{ref}}}|}{|V_c|}$$

3. **ドメイン整合性 (Domain Consistency)**： S_{ref} の式のうち、 c と同じドメインに属するものの割合。

$$\text{Dom}(c) = \frac{|\{s \in S_{\text{ref}} : \text{domain}(s) = \text{domain}(c)\}|}{|S_{\text{ref}}|}$$

これらを単発特徴量と合わせた 10 次元ベクトルを多層パーセプトロンに入力し、スコアを出力する。

4. 結果と考察

4.1 データセット

■4.1.1 数式データベース

- **総式数** : 3,244 式
 - 論文 PDF 由来 : 2,574 式 (39 本の PDF から Gemini API で抽出)
 - 教科書・ハンドブック由来 : 670 式 (化学工学標準分野から大規模言語モデルで生成)

- **ドメイン分布** : 813 種類の細粒度ドメイン (一部を表に)

ドメイン	式数
State-Space Models	52
CSTR Kinetics	49
Laplace Transform	43
Biodiesel Transesterification	42
Helium Core Heat Exchanger Modeling	40
Routh Stability Analysis	38
Bioprocess Kinetics	34

- **1 式あたりの変数数** : 平均 5.2 (中央値 5、範囲 0~29)
 - 分布 : 3 変数 502 式、4 変数 649 式、5 変数 573 式、6 変数 508 式、7 変数以上 720 式 (式の多くが 3~6 変数)

■4.1.2 訓練・評価ケース 1,107 件の訓練ケースを、大規模言語モデル (Gemini 2.5 Pro) による生成 と プログラムによる拡張を組み合わせて構築した :

バリエント	件数	生成方法
original	93	大規模言語モデルがプロセス記述と正解式集合を直接生成
context_paraphrased	279	テンプレートで文脈文を3通りに言い換え（大規模言語モデル不使用）
random_io_from_models	733	originalの正解式から入出力を再割当（自由度ゼロを保証；大規模言語モデル不使用）
swap_io	2	originalの入出力を入れ替え（自由度解析を通過したもののみ；大規模言語モデル不使用）

- 1 ケースあたり入力変数数：平均 4.8（範囲 2～18）

- 1 ケースあたり出力変数数：平均 1.4（範囲 1～4）
- 正解式数の分布：1 式正解 56.1%、2 式正解 36.4%、3 式以上正解 7.5%

original バリエーションの生成プロンプト（Gemini 2.5 Pro に投げる本文；実装の英語指示をそのまま示す）：

You are helping to create training data for a graph neural network that selects relevant equations for physical process modeling.

You are given a catalog of equations. Each equation has:

- model_id: a unique ID in the form "<source_id>__<eq_id>"
- domain: a short domain label
- variable_symbols: list of variable symbols in the equation

CATALOG ({n_eq} equations, domains: {domain_list}):
{catalog_text}

TASK:

Using ONLY the models listed above, create approximately {n_cases} realistic and physically meaningful core training cases.

For each core case:

- Pick one or more model_ids that together represent a coherent physical model.
- Define a context string describing the physical scenario in natural language.
- Choose input_variables and output_variables from variable_symbols of selected models.
- correct_model_ids: list of model_ids forming a correct mathematical model.

CONDITIONS FOR correct_model_ids:

- (1) Given input_variables values and the equations, it MUST be possible to solve for all output_variables.

(2) The equation set MUST be sufficient and minimal (no redundant equations).

REQUIREMENTS:

- All correct_model_ids MUST exactly match entries from the catalog.
- input/output variables MUST be subsets of variable_symbols from selected models.
- Make cases diverse across the available domains.
- Generate at least {n_cases} cases.

OUTPUT: JSON object with "cases" array, each having:

case_id, context, input_variables, output_variables, correct_model_ids

{n_eq} / {domain_list} / {catalog_text} / {n_cases} は実行時に式データベースから埋め込む。プロンプト設計上の重要なポイント：

- **閉世界制約**：「Using ONLY the models listed above」で、カタログ外の架空の式を生成させない（ハルシネーション抑止）。
- **自由度条件の明示**：「it MUST be possible to solve for all output_variables」で、物理的に可解な入出力ペアを要求。
- **最小性**：「sufficient and minimal」で冗長な式を正解集合に含ませない。
- **ドメイン多様性**：「Make cases diverse across the available domains」でカタログの広範なドメインから選ばせる。
- **構造化出力**：OUTPUT 指定で JSON 形式に固定し、後段のパースが決定論的に扱えるようにする。

context_paraphrased バリエーションのテンプレート（大規模言語モデル不使用、3 種類の前置き文を機械的に付与）：

"This case describes the following physical model: {context}"

"In this scenario, the underlying process can be summarized as: {context}"

"From the perspective of process modeling, this case focuses on: {context}"

random_io / swap_io バリエーション：大規模言語モデルは使わず、アルゴリズム的に生成する。

original の correct_model_ids をそのまま引き継ぎ、その変数集合から入出力を再割当する。割当後、各ケースについて**自由度解析**（既知の入力 + 式数 = 未知の出力数）を走らせ、自由度ゼロ（物理的に可解）のものだけを採用。これにより、不整合なケースを構築時に除外している。

なぜ original だけ大規模言語モデルを使うか：API 呼び出しコストと再現性を考慮し、大規模言語モデルは「種ケース」生成（文脈とそれらに対応する正解式集合の意味的対応付け）という言語モデルでしか書けない部分に限定して用い、そこから派生するバリエーション（言い換え・入出力入れ替え）は全て決定論的なプログラムで展開する設計とした。

■4.1.3 学習・評価の分割 データは**文献単位**（source_id）で訓練・検証・テストに分割する（60% / 20% / 20%）。これにより、訓練とテストで同一文献の数式が混在することを防ぎ、新規文献に対する汎化性能を評価する。

素朴な「ケース単位ランダム分割」では、同じ文献由来のケース（同じ正解式、類似した文脈）が訓練とテスト両方に入り、モデルは見覚えのある式を引くだけで高スコアを得られる（データリーク）。**文献単位**で分割することで、テストは訓練で一度も見えていない source に対する性能となり、本研究の実運用（新しい論文が追加されたときに正しい式を見つける）に直結する汎化性能を評価できる。

4.2 評価指標

- **MRR_first** (**Mean Reciprocal Rank** of first correct)：各ケースについて、ランキング中で最初に現れた正解式の順位の逆数を取り、全ケースで平均する。ランキング対象数 $N = 3,244$ に対し、理論上の値域は $[1/N, 1] \approx [3 \times 10^{-4}, 1]$ 、1 に近いほど良い。全ケースで正解が 1 位にくる場合に 1、全ケースで最下位にくる場合に $1/N$ となる。
- **FullRecall@K**：ランキング上位 K 件に**全ての正解式**が含まれるケースの割合（複数式正解を重視）。値域は $[0, 1]$ 、1 に近いほど良い。
- **MAP** (Mean Average Precision)：情報検索の標準指標。「**全ての正解式がランキングで前方にまとまって現れているか**」を連続値で測る。

ケース q の正解式集合を R_q 、ランキング上位 k 件に含まれる正解の割合 (precision at k) を

$$P_q(k) = \frac{\text{上位 } k \text{ 件中の正解数}}{k}$$

と定義する。このとき Average Precision（単ケースの AP）は**正解が現れた順位だけ**で平均を取る：

$$\text{AP}(q) = \frac{1}{|R_q|} \sum_{k=1}^N P_q(k) \cdot \mathbb{1}[\text{順位 } k \text{ の式} \in R_q]$$

MAP は全ケースに対する AP の平均： $\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$ 、値域は $[0, 1]$ 、1 に近いほど良い。

直感：「正解式が新たに現れた順位 k において、『その時点までに上位に正解が何個詰まっているか』を $P_q(k)$ で記録する。全ての正解が前方にまとまっているほど $P_q(k)$ が大きく、AP が高くなる。」

簡単な例：ケース q の正解が 3 式 ($R_q = \{a, b, c\}$)、ランキングが $[a, x, b, y, c, \dots]$ （順位 1, 3, 5 が正解、 x, y は不正解）の場合：

順位 k	上位 k 中の正解	$P_q(k)$	加算？
1	$\{a\}$	$1/1 = 1.00$	Yes (a)
2	$\{a\}$	$1/2 = 0.50$	No
3	$\{a, b\}$	$2/3 \approx 0.67$	Yes (b)
4	$\{a, b\}$	$2/4 = 0.50$	No
5	$\{a, b, c\}$	$3/5 = 0.60$	Yes (c)

$\text{AP}(q) = (1.00 + 0.67 + 0.60)/3 \approx 0.756$ 。全正解が 1, 2, 3 位に詰まっていれば $\text{AP} = (1 + 1 + 1)/3 = 1$ となる。

3 指標の違い：MRR_first は「最初の正解が何位か」だけを見て、他の正解の位置は無視する。FullRecall@ K は「上位 K 件に全正解が揃ったか」の 0/1 判定で、境界条件に敏感。MAP は **全正解の順位**を連続値で評価し、単発正解にも複数正解にも使える情報検索の標準

指標である。

4.3 主要な結果

ランダムシード 10 通りで評価した平均値：

手法	MRR_first	FullRecall@3	MAP
ベースライン (Stage 1 スコアのみ)	0.937	0.826	0.936
リランカー (基本 7 次元特徴量のみ)	0.959	0.881	0.953
リランカー + Set-aware 3 特徴量 (10 次元)	0.980	0.904	0.974

■統計的有意性 同じ実験でもランダムシード (初期重み・データシャッフル) を変えると結果は揺らぐため、観察された差が**偶然のばらつきなのか、手法の改善なのか**を判定する必要がある。本研究では **対応あり t -検定** (paired t -test) を用い、各シードでベースラインと新手法を走らせてシードごとの差 d_i を取り、 t 値と p 値を計算した。 p 値は「もし実際には差がなかったとしたら、観察された以上の差が偶然に生じる確率」を意味する ($p < 0.05$ を有意、 $p < 0.01$ を高度に有意とするのが慣例)。

Set-aware 特徴量導入の効果について、10 シード間の対応あり t -検定を行った：

指標	t 値	p 値 (両側)	有意性
MAP	3.46	$p = 0.004$	高度に有意
複数正解式ケースの FullRecall@3	2.12	$p = 0.032$	有意
複数正解式ケースの MRR_worst	2.28	$p = 0.024$	有意
MRR_first (全体)	1.28	$p = 0.231$	有意ではない

単発評価 (MRR_first) では有意差はないが、**複数式の組合せ評価では明確な改善**が認められる。

4.4 考察

個別の式選択 (MRR_first) では、ベースライン (Stage 1 スコアのみ) が既に 0.937 と高く、グラフ構造特徴量を追加しても改善幅は +0.043 (ベースライン比 +4.6%, $p = 0.231$ で有意差なし) にとどまった。しかし、**複数式を同時に選ぶ課題** (複数正解ケースの FullRecall@3 · MRR_worst) では、Set-aware 特徴量の導入により +6.5%, +5.5% の改善が得られ、それぞれ $p = 0.032, p = 0.024$ で統計的に有意であった。

この知見は、グラフニューラルネットワーク的アプローチの有効性に関する示唆を与える：

単一対象のスコアリングではなく、対象集合の整合性評価こそ、グラフ構造の情報が差別化を生む場面である。

4.5 今後の展望

A. 変数レベルの意味類似度の導入

現状の文脈情報利用は次の通り：

情報の粒度	現在の扱い
式全体の文脈 (式テキスト、周辺説明文、ドメイン名)	○ 利用済み (Stage 1 の TF-IDF、Stage 2 の f0, f2)
変数単位の意味 (各変数の自然言語記述)	× 未使用 (記号の完全一致のみ)

変数マッチングを記号一致から意味ベースに拡張することで、表記揺れ (c_p vs C_p など) を吸収し、変数共有エッジの欠落を減らす方針。

B. 式データベースの拡大

次の 2 方向を分けて検討する：

- **単一ドメイン内の数式の増強**：既存ドメインの網羅性を高める
- **ドメイン拡張**：新規分野 (例：バイオプロセス応用、半導体プロセス等) の追加



C. 実プロセスでのケーススタディ

実際のプロセス設計・制御設計の業務に本手法を適用し、実用性と限界を評価する。

5. まとめ

1. **目的**：物理モデル構築の手作業部分を自動化し、設計・制御・最適化・安全解析の基盤を提供する
2. **関連研究**：シンボリック回帰は実験データ依存、大規模言語モデル直接生成はハルシネーションと組合せ整合性の問題。本研究は実在数式からの**数式検索**に定式化することで、両者の欠点を回避する
3. **手法**：二部グラフ（式 3,244 個、変数 3,700 個）上で 2 段階の数式検索を行う。Stage 2 の多層パーセプトロン（MLP）リランカーに、**Set-aware なグラフ特徴量**を組み込む
4. **結果**：単発選択（MRR_first）では従来手法で十分だが、**複数式の組合せ選択**（FullRecall@3, MAP）で Set-aware 特徴量が統計的に有意な改善を示した（ $p = 0.004 - 0.032$ ）
5. **展望**：変数レベルの意味類似度導入、データ規模の拡大方針の分離検討、実プロセスでのケーススタディ